U-Net Architecture & Model Building

A complete guide to image segmentation with U-Net — from architecture to TensorFlow implementation. Covering the full pipeline: theory, design, and code.

IMAGE SEGMENTATION | DEEP LEARNING | TENSORFLOW

Made with GAMMA

What is U-Net?

U-Net is a convolutional neural network designed primarily for **image segmentation** — dividing an image into meaningful parts to identify specific objects, such as separating a tumor from healthy tissue in a medical scan.

- Introduced in 2015 by Ronneberger, Fischer, and Brox
- Named after its distinctive **U-shaped architecture** when drawn
- Excels in **medical imaging** with limited labeled data
- Combines local and global context for precise pixel-level predictions

Task

Pixel-level classification — assign every pixel a label

Strength

High accuracy even with very few training images

Made with GAMMA

The U-Shaped Architecture

U-Net's architecture is divided into three core regions that work together to encode context and decode precise spatial location.

Contracting Path

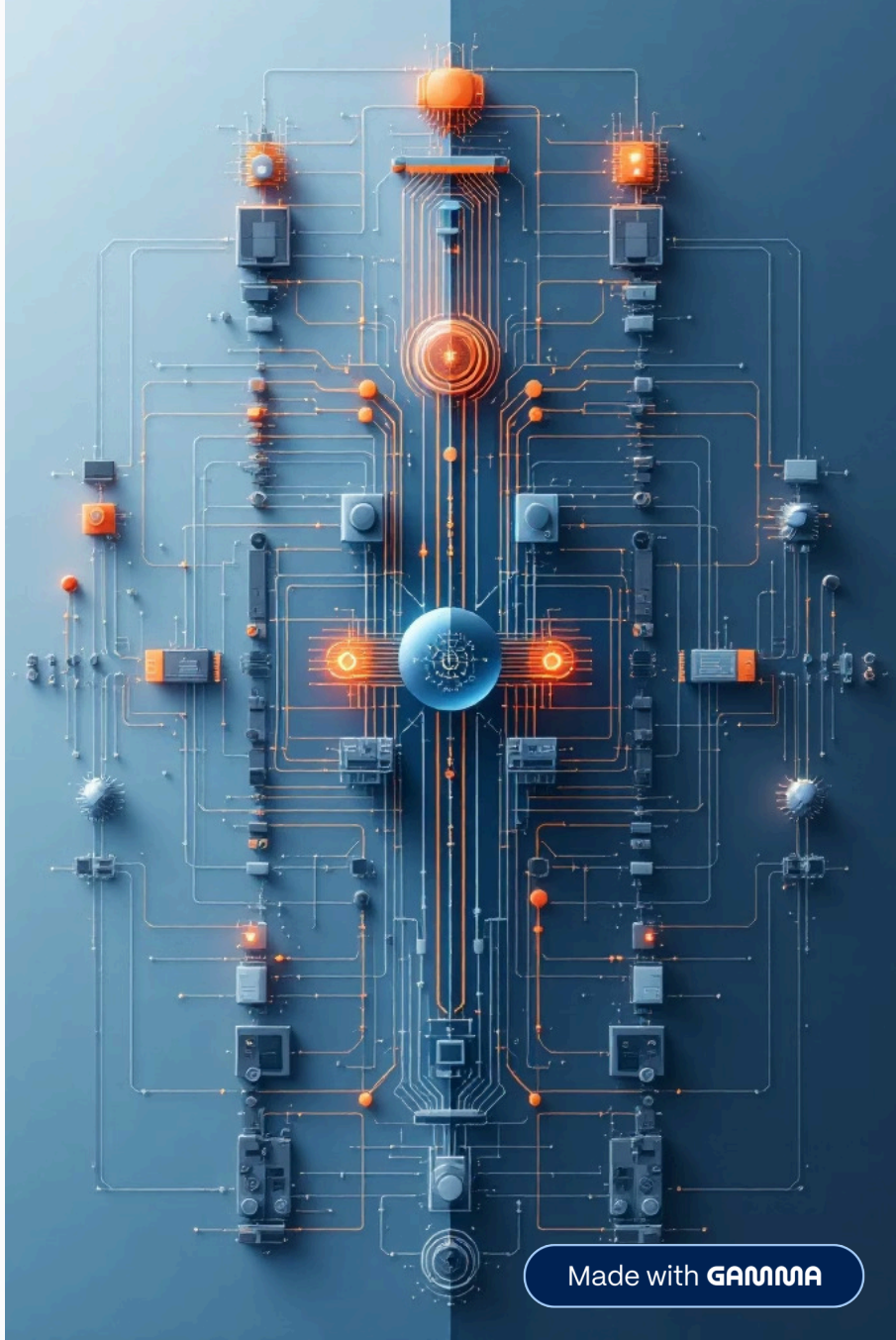
Encoder — captures context by progressively reducing spatial dimensions

Bottleneck

Bridge — learns the deepest feature representations

Expanding Path

Decoder — reconstructs spatial resolution for precise localization



Made with GAMMA

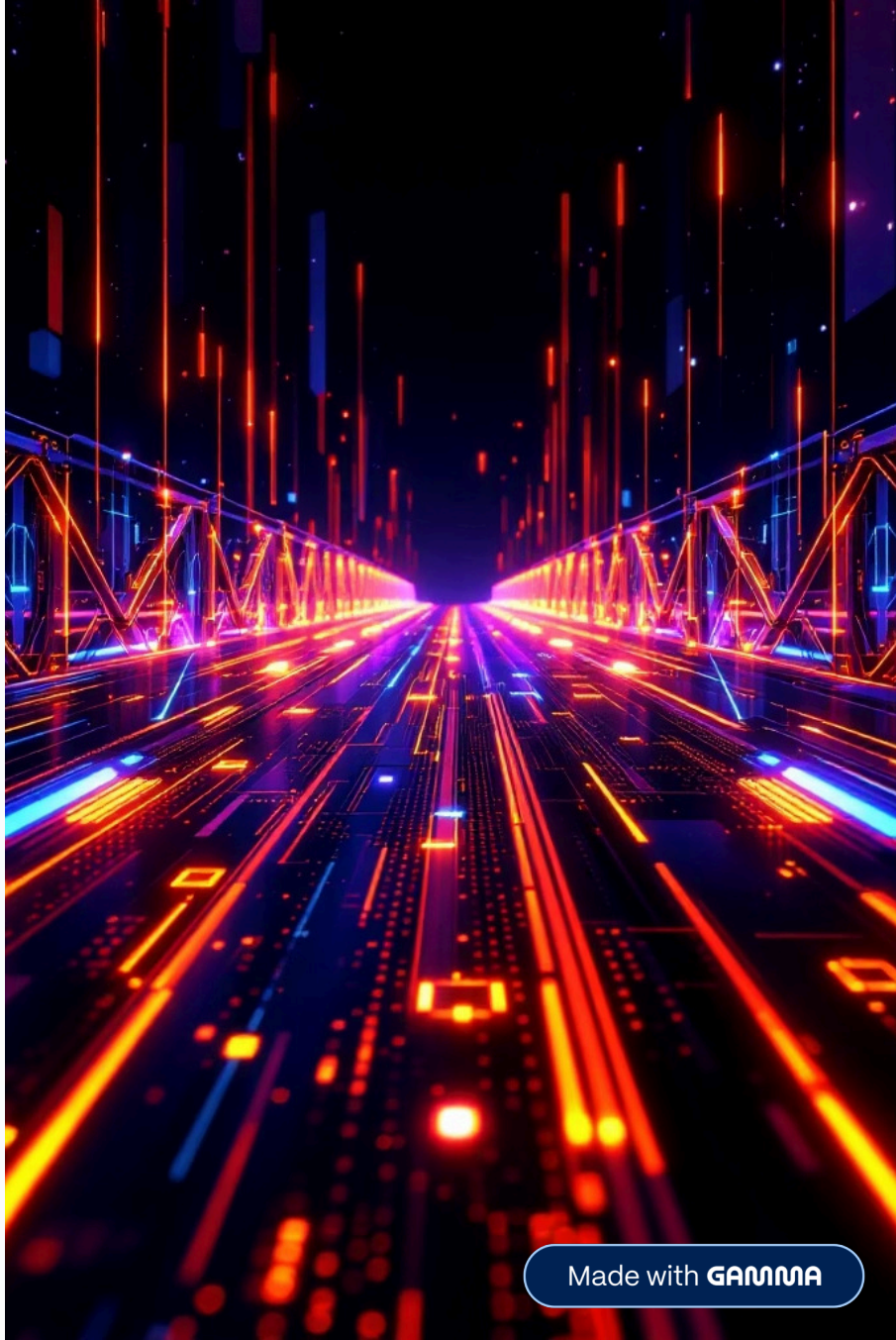
Skip Connections: The Secret Weapon

The Problem They Solve

During downsampling, fine spatial details — like edges and boundaries — are inevitably lost. Skip connections **directly link encoder layers to their decoder counterparts**, preserving this critical spatial information.

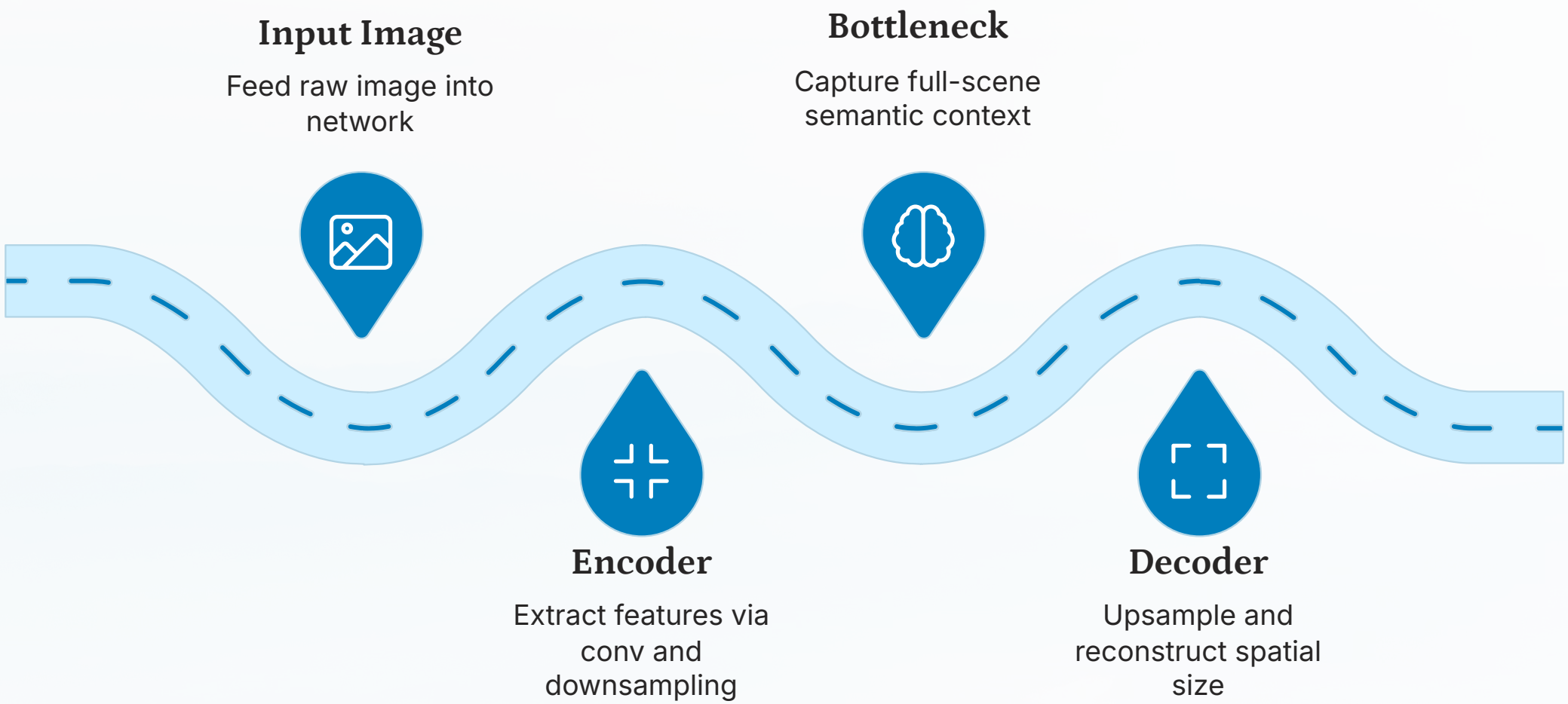
How They Work

Feature maps from the encoder are **concatenated** with the corresponding upsampled feature maps in the decoder. This fusion of deep semantic features with shallow spatial features is what makes U-Net exceptionally accurate.



Made with GAMMA

How U-Net Processes Data — Step by Step



At each decoder stage, skip connections fuse fine-grained spatial details from the encoder with high-level semantic context from the decoder — enabling sharp, accurate boundary predictions at full resolution.

Made with GAMMA

Implementation — Encoder Block

```
import tensorflow as tf

def encoder_block(inputs, num_filters):
    x = tf.keras.layers.Conv2D(
            num_filters, 3, padding='valid')(inputs)
    x = tf.keras.layers.Activation('relu')(x)

    x = tf.keras.layers.Conv2D(
            num_filters, 3, padding='valid')(x)
    x = tf.keras.layers.Activation('relu')(x)

    x = tf.keras.layers.MaxPool2D(
            pool_size=(2, 2), strides=2)(x)
    return x
```

Key Design Choices

- **Two 3×3 Conv2D layers** — each followed by ReLU activation to add non-linearity
- **padding='valid'** — no zero-padding; spatial dimensions shrink after each conv
- **Filter doubling** — 64 → 128 → 256 → 512 across levels
- **MaxPooling (2×2)** — halves spatial size, forces the network to learn abstract features

Made with GAMMA

Implementation — Decoder Block

```
def decoder_block(inputs, skip_features, num_filters):
    x = tf.keras.layers.Conv2DTranspose(
        num_filters, (2, 2),
        strides=2, padding='valid')(inputs)

    skip_features = tf.keras.layers.Resizing(
        x.shape[1], x.shape[2])(skip_features)
    x = tf.keras.layers.Concatenate()([x, skip_features])

    x = tf.keras.layers.Conv2D(
        num_filters, 3, padding='valid')(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = tf.keras.layers.Conv2D(
        num_filters, 3, padding='valid')(x)
    x = tf.keras.layers.Activation('relu')(x)
    return x
```

The Skip Connection Magic

Conv2DTranspose

Doubles spatial dimensions — the learnable inverse of pooling

Resizing + Concatenate

Aligns and merges encoder's high-res detail with decoder's semantic context

Two Conv + ReLU

Refines the combined feature map for the next upsampling stage

Made with GAMMA

Implementation — Full U-Net Model

```
def unet_model(input_shape=(256, 256, 3), num_classes=1):
    inputs = tf.keras.layers.Input(shape=input_shape)

    # Contracting Path (Encoder)
    s1 = encoder_block(inputs, 64)
    s2 = encoder_block(s1, 128)
    s3 = encoder_block(s2, 256)
    s4 = encoder_block(s3, 512)

    # Bottleneck
    b1 = tf.keras.layers.Conv2D(1024, 3, padding='valid')(s4)
    b1 = tf.keras.layers.Activation('relu')(b1)
    b1 = tf.keras.layers.Conv2D(1024, 3, padding='valid')(b1)
    b1 = tf.keras.layers.Activation('relu')(b1)

    # Expansive Path (Decoder)
    d1 = decoder_block(b1, s4, 512)
    d2 = decoder_block(d1, s3, 256)
    d3 = decoder_block(d2, s2, 128)
    d4 = decoder_block(d3, s1, 64)

    # Output Layer
    outputs = tf.keras.layers.Conv2D(
        num_classes, 1, padding='valid', activation='sigmoid')(d4)

    model = tf.keras.models.Model(inputs=inputs, outputs=outputs, name='U-Net')
    return model

model = unet_model(input_shape=(572, 572, 3), num_classes=2)
model.summary()
```

The encoder skip connections `s1–s4` are passed directly to the corresponding decoder blocks, ensuring fine spatial detail is preserved at every resolution level.

Made with GAMMA

Why U-Net Works So Well

Skip Connections

Preserve spatial detail lost during downsampling — the secret to sharp boundary detection

Symmetric Design

Encoder depth mirrors decoder depth, ensuring balanced feature propagation at every level

Small Dataset Friendly

Architecture + data augmentation compensates for limited labeled training samples

Fully Convolutional

No fully connected layers — runs on any image size in a single efficient forward pass

Broad Applications

Medical imaging, satellite analysis, autonomous driving, and even Stable Diffusion's internal denoising

Made with GAMMA